AKS Backup and Restore: Presenter Script

Follows the slides in AKS-Backup-Restore-v2.pptx, in order. About 18 minutes.First person, ready to read. Grey italics are notes for you, not for reading out.

Slides 1 and 2: Opening and agenda

Hi all. Today I will walk you through AKS backup and restore on the IHUB platform. First how it works, then how we built it, then permissions, and then a step by step restore in the portal, with the things that trip people up.

The idea is simple. We take scheduled backups of the cluster, and if something goes wrong we can bring it back, either into the same cluster or into a separate restore cluster, without rebuilding anything by hand.

Slide 3: How AKS Backup works

Three ideas to remember. First, the backup extension. It is a component installed inside the cluster. The Backup vault talks to the cluster only through this extension, for both backup and restore.

Second, Trusted Access. This is how the vault reaches the cluster, with a least privilege role. We do not hand broad admin rights to the vault.

Third, the operational tier. Backups stay in our own subscription, in the backup storage account and the snapshot resource group. This is the only tier we use. It backs up the cluster resources. Volume snapshots are off, so volume data is not included.

Slide 4: Our architecture

Read this left to right. The original cluster has the backup extension. The vault reaches it through Trusted Access, and the backup data goes to a storage account and container.

The storage account has a private endpoint, and that is created by the cloud team. This one matters: without the private endpoint the storage account cannot talk to the cluster and backups fail.

On the right is the restore cluster, the one with the rst suffix. Restoring to it is an alternate cluster restore, so the original cluster is never touched.

Slide 5: What backup.tf creates, and how it works

One file, backup.tf, in the platform module builds the whole backup setup. Let me go through it in the order it is written.

It starts with locals. It reads each backup list from the instance conf.yaml, and if a list is missing it falls back to an empty list, so an environment with no backup blocks creates nothing. Then it turns each list into a map. Storage accounts, vaults and policies are keyed by name. Extensions and backup instances are keyed by cluster name. Every resource below uses for_each on one of these maps, so one entry in conf.yaml means one resource. Retention is also a local: 90 days if the environment is prd, otherwise 7.

The backup instance map has a useful detail. In conf.yaml you only give the cluster and the policy. The code finds the vault from the policy, and the storage account from that cluster's extension entry, so you do not repeat them.

Now the resources. The storage account is StorageV2, Standard, ZRS and Cool tier, TLS 1.2 only, no public blob access, and public network access is off, which is why it needs the private endpoint.

Versioning and change feed are on, and deleted blobs and containers are kept for 7 days. The container is private.

The backup extension is named azure-aks-backup, type microsoft.dataprotection.kubernetes, installed into the dataprotection-microsoft namespace. Its settings tell it which container, storage account, resource group, subscription and tenant to use. It has an explicit depends_on for the cluster, the storage account and the container, so it never installs before they exist.

The Backup vault is zone redundant, with soft delete always on and immutability set to Locked, and it has a system assigned identity. That identity is the one that receives the roles later. Locked immutability cannot be switched off afterwards, so be careful with it. A separate management lock, CanNotDelete, is placed on each vault.

The policy is named bp- plus the policy name. It runs in UTC, repeats every 4 hours, and keeps backups in the operational store for the retention days I mentioned. The backup instance is named bi- plus the cluster name. It links the vault, the cluster, the policy and the snapshot resource group, and its parameters say full cluster: cluster scoped resources on, and no included or excluded namespaces or resource types. It depends on the extension, because the extension must be running first.

Finally the access. A Trusted Access role binding gives the vault the backup-operator role on the cluster. Then the role assignments from slide 10: the vault identity gets Reader on the cluster and the snapshot resource group, Disk Snapshot Contributor and Data Operator for Managed Disks on the snapshot resource group, and Storage Blob Data Contributor on the storage account. The extension identity gets Storage Account Contributor on the storage account, and the cluster identity gets Contributor on the snapshot resource group. These access resources are created per backup instance, so per protected cluster.

The order is not written as one long depends_on list. Terraform works it out from the references: storage account, container, extension, vault, instance, then access. Only the extension and the instance have an explicit depends_on.

Slides 6 and 7: How conf.yaml drives it

Nothing is hard coded. backup.tf reads lists from conf.yaml and creates one resource for each entry. A local is just a named helper value that Terraform works out once and reuses. It is not a setting you pass in.

So if you add an entry to a list, the next pipeline run creates the matching resources. On slide 7 you can see the five blocks: storage account, backup vault, backup extension, backup policy and backup instance, and what each one creates.

Here is what you actually write. storage_account takes a name and a container name. backup_vault takes a name. aks_backup_extension takes the cluster, the storage account and the container. backup_policy takes a name and the vault. backup_instance takes the cluster and the policy. For SIT that is the storage account sasbfhaksihubsitbackup, the container blobakssitbackup, the vault bv-bfhaks-ihub-eus2-sit-01 and the policy aks-4h-backup-7d-retention, which Terraform names bp- followed by that.

One thing to know. Interval, retention and full cluster backup are not read from conf.yaml. They are set in backup.tf, so changing them in conf.yaml does nothing. Also, in Dev, every rst entry is commented out in conf.yaml. The rst cluster gets an extension but no backup instance, so it is a restore target and is not backed up itself.

Slide 8: How workload onboarding works, and what we changed

First, how workload onboarding works normally, because the rst changes sit inside it. Workload onboarding is how an application team gets a namespace on a cluster. Each workload has one yaml file in the use case folder. For example the Profile API file says its name and version, its support group and data classification, whether it uses workload identity, whether Istio injection is on, its Prometheus alert rules, and a list of platform instances. For each instance it names the app team group, its role, and whether alerts are on.

It runs in two phases. Phase 1 runs once per platform instance, for example ihub-dev-eus2. It reads the instance conf.yaml, finds the use case, reads every workload file, and keeps only the workloads that list this instance. Then for each workload it creates a user assigned managed identity, one per component that uses workload identity. It gives each app team group the Cluster User role on the resource group, so they can run az aks get-credentials. And it creates Prometheus alert rule groups for workloads that have alerts on, one for each cluster. It also outputs the cluster names and the identities, which phase 2 needs.

Phase 2 runs once per cluster, using the cluster names from phase 1. For each workload it creates the Kubernetes namespace, with the Istio label and the support group and data classification labels. It gives the app team group and the pipeline credential their roles on that namespace. It creates a service account for each component, annotated with the identity's client id. And it creates a federated credential, which links that service account to the managed identity using the cluster's OIDC issuer. That is what lets a pod sign in to Azure without a secret. Last, it updates the Istio gateway with the new hosts.

Now our three changes, so the rst cluster works without touching the original one. In phase 1, any cluster with rst in its name is left out of the Prometheus alert list. Before that, the alert rule names clashed and the pipeline failed.

In phase 2, we replaced a hard coded environment mapping with detection from conf.yaml. Any cluster with rst in the name is treated as a restore target, and the code reads its OIDC issuer.

In the module, we add one extra federated credential per workload identity for the rst cluster. Same identity, same subject, different issuer. Without it, pods on the rst cluster cannot get a token and stay stuck in Init. We only add, we never replace, and the plan for the original cluster showed no resource changes.

Slide 9: Restore options

You can do a full restore, or an item level restore where you pick namespaces, labels or specific kinds. You can restore to the original cluster, where existing resources with the same name must be deleted first or patched, or to an alternate cluster. The alternate cluster is how we validate with the rst cluster.

Slide 10: Permissions

This table is what backup.tf grants: the cluster identity gets Contributor on the snapshot resource group, the vault gets Reader on the cluster and the snapshot roles, and Storage Blob Data Contributor on the storage account. The extension identity gets Storage Account Contributor on the storage account, and Trusted Access uses the backup-operator role.

These are created only for clusters that have a backup instance, so only the original cluster. The rst cluster does not have one, so when you validate a restore it will ask you to grant them.

Note: the slide flags that Microsoft's guidance lists Storage Blob Data Contributor for the extension identity, while our code gives it Storage Account Contributor. Say it as a thing to check if a restore fails on storage permission.

Slides 11 to 14: Restoring in the portal

Now the restore itself, in six moves, and it all starts from the Backup vault of the environment. Use the vault for the environment you are restoring: dev, sit, uat and so on. The target cluster must have the backup extension installed and Trusted Access enabled.

Steps one to four are on slide 13. Search for Backup vaults and open yours. Open the AKS backup instance, or go through Resiliency, Recover, Azure Kubernetes Services. Click Restore, and pick the restore point, which is the point in time you want.

Steps five to eight are on slide 14. Select the target cluster, for example the rst cluster. Select the namespaces and resources. Click Validate. For the rst cluster, backup.tf does not create Trusted Access or the roles, so expect Validate to ask for them. Grant what it asks for and validate again until it passes. Then review, click Restore, and follow progress under Backup jobs.

Note: slide 11 is just a divider. If people are hands on, open the portal here and show the Restore screen.

Slide 15: Choosing what to restore

You can filter by namespace, cluster scoped resources, labels, API groups and kinds, and you can map a namespace to a different one, which is created if missing. If a resource already exists, the default is Skip, which leaves it as it is, so delete it first if you want it restored. Patch updates the mutable fields, for example the replica count.

Slides 16 and 17: Verify and things that trip people up

After the restore, use kubectl to check pods and deployments in the namespace, follow the restore job in the vault until it completes, and test the workloads before you call it a success.

Six traps. Backups do not delete and recreate, so delete the existing resource before restoring in place. Missing namespaces are created, and namespace mapping lets you restore next to the original. Restore into the same region, because cross region needs vault tier, which we do not use. Restore does not add nodes, so make sure the target cluster has capacity. Same names are skipped by default, so choose Patch to update them. And the extension and Trusted Access must be there on the target.

Slide 18: Our rollout, and the prerequisites

This is the order for a new environment. Before anything goes into conf.yaml, there are prerequisites that are not in our code.

First, resource providers. In the subscription, check that Microsoft.DataProtection and Microsoft.KubernetesConfiguration are registered. If either is not, we request it, because we do not have the rights to register it. Do this first, since it can take days.

Second, the storage account. Names are 3 to 24 characters, lowercase letters and numbers only, and it must be in the same region as the cluster. It has to exist before the vault and the extension. If someone created it by hand, as in SIT, import it into Terraform first.

Third, request the cloud team to create the private endpoint for that storage account. Raise this as soon as the account exists, because backups do not work until it is in place.

After that: conf.yaml with the backup settings and the optional rst cluster, then the pipeline deploys everything, workload onboarding picks up the rst cluster automatically, and we verify the vault and the extensions.

Note: the deck does not list the resource provider step yet. Say it out loud here, or add it to slide 18.

Where we are with SIT

For SIT, the conf.yaml change is merged. The extension installed and the dataprotection namespace is there. The first apply failed on the backup policy with a not found error for the vault. The policy takes the vault name as plain text, so Terraform does not know it must wait for the vault. The fix is one line: read the vault name from the vault resource itself. The name is the same, so existing environments show no change.

Still to do in SIT: the private endpoint from the cloud team, confirm the cluster shows as protected, and run a first backup.

Slide 19: Summary and close

To summarise. The vault reaches the cluster through the backup extension and Trusted Access. A restore starts in the vault of the environment: Restore, restore point, target, resources, validate. Grant the roles validation asks for, then restore, and verify with kubectl and the restore job.

The Microsoft Learn links are on the slide. If you own an app, there is nothing to change, because your namespace is covered when the cluster is. If you need something restored, tell us the namespace and the time, and remember non-production keeps only seven days. Suggestions are welcome. Questions?